Practical 1

Sudo apt update

Sudo apt install git

STEP 2: Configure Git (First Time Setup)

In Command Prompt:

git config --global user.name "Your GitHub Username"

git config --global user.email "your_github_email@example.com"

Verify:

git config –list

STEP 3: Create Data Science Project Folder

Create a project folder:

mkdir DataScienceProject

cd DataScienceProject

Now create a sample file (for testing):

gedit Data Science Project.py

If you have Python files, you can copy them inside this folder.

STEP 4: Initialize & clone Git Repository

Inside your project folder:

git init

You will see:

Initialized empty Git repository

Now check status:

git status

Now clone the git with local system :

git clone “urlofnewlycreatedrepo”

STEP 5: Create Repository on GitHub

1. Go to https://github.com
2. Click **New Repository**

3. Repository name: DataScienceProject
4. Select:
    - Public (or Private)
    - initialize with README
5. Click **Create Repository**

GitHub will show commands. We will use them.

STEP 6: Add Files & Commit

Add all files:

git add Data Science Project.py or

git add .

Commit changes:

git commit -m "Initial commit for data science project"

STEP 7: Connect Local Repo to GitHub

> Copy your repository URL.

It will look like:

https://github.com/yourusername/DataScienceProject.git

> Now in Command Prompt:

git remote add origin https://github.com/yourusername/DataScienceProject.git

> Set main branch:

git branch -M main

> Push project:

git push -u origin main

STEP 9: Generate GitHub Personal Access Token

1. Go to GitHub
2. Click profile → Settings
3. Developer Settings
4. Personal Access Tokens

5. Tokens (classic)
6. Generate new token
7. Select:
    - repo (full control)
8. Generate
9. Copy the token

When pushing:

- Username → your GitHub username
- Password → paste token

Whenever you make changes:

```
git status
git add .
git commit -m "Describe your changes"
git push
```

Practical 2

```
Sudo apt update
Sudo apt install git
```

STEP 2: Configure Git (First Time Setup)

In Command Prompt:

```
git config --global user.name "Your GitHub Username"
git config --global user.email "your_github_email@example.com"
```

Verify:

```
git config –list
```

STEP 3: Check Current Branch

```
git branch
```

You will see:

```
* main
```

STEP 4: Create Feature Branch

Example: create branch called feature-add-function

git branch feature-add-function

> Switch to it:

git checkout feature-add-function

OR shortcut:

git checkout -b feature-add-function

Check:

git branch

Now * will be on feature branch.

STEP 5: Work in Feature Branch

> Edit file:

gedit multiply.py

> Now:

git add .

git commit -m "Added multiply function"STEP 1: Check Current Branch

git branch

You will see:

* main

STEP 6: Create Feature Branch

Example: create branch called feature-add-function

git branch feature-add-function

> Switch to it:

git checkout feature-add-function

OR shortcut:

git checkout -b feature-add-function

Check:

git branch

Now * will be on feature branch.

STEP 7: Work in Feature Branch

> Edit file:

gedit multiply.py

Now:

git add .

git commit -m "Added multiply function"

> Push branch to GitHub:

git push -u origin feature-add-function

STEP 8: Switch Back to Main

git checkout main

STEP 9: Merge Feature Branch

git merge feature-add-function

If no conflict → success

Push:

git push

>>> NOW IMPORTANT PART — CREATE MERGE CONFLICT

To show conflict, do this:

Step A: Create New Branch

git checkout -b feature-change

Edit same file:

gedit multiply.py

Commit:

git add .

git commit -m "Modified multiply logic in feature-change"

Step B: Switch to Main and Modify Same Line

git checkout main

Edit file again:

gedit multiply.py

Commit:

git add .

git commit -m "Modified multiply logic in main"

Step C: Merge → Conflict Happens

git merge feature-change

Now you will see:

CONFLICT (content): Merge conflict in multiply.py

STEP 10: Resolve Conflict

Open file:

gedit multiply.py

Example:

```
def multiply(a, b):
    return a * b
```

Delete all <<<<, ====, >>>> lines.

Save file.

Now:

git add multiply.py

git commit -m "Resolved merge conflict"

Push:

git push

Practical 3

STEP 1: Open Original Open-Source AI Project

Original Repository:

https://github.com/JerryKurata/MachineLearningWithPython

Open the repository in browser.

Click on the "Fork" button (top right corner).

Select your GitHub account and create fork.

This creates a copy in:

https://github.com/NikitaLokhande25/MachineLearningWithPython

STEP 2: Configure Git (Only First Time)

Command:

```
git config --global user.name "Nikita Lokhande"
git config --global user.email "nikitalokhande@gmail.com"
```

This sets Git username and email.

STEP 3: Clone Forked Repository

Command:

```
git clone https://github.com/NikitaLokhande25/MachineLearningWithPython.git
```

This downloads the forked repository to local system.

STEP 4: Enter Project Directory

Command:

cd MachineLearningWithPython

This moves into the project folder.

STEP 5: Create Feature Branch

Command:

git checkout -b fork-practical

This creates and switches to a new branch named fork-practical.

STEP 6: Create New File

Command:

nano fork_practical.txt

Add some content inside file.

Save and exit (Ctrl+X → Y → Enter).

This creates a new file for changes.

STEP 7: Add Changes to Staging Area

Command:

git add .

This stages all modified files.

STEP 8: Commit Changes

Command:

git commit -m "Added fork practical file"

This saves changes locally.

STEP 9: Push Branch to GitHub

Command:

git push origin fork-practical

Enter GitHub username and Personal Access Token if asked.

This uploads branch to GitHub.

STEP 10: Create Pull Request

Open your forked repository on GitHub.

Click "Compare & pull request".

Add:

Title: Added fork practical file

Description: Fork practical implementation

Click "Create Pull Request".

This submits the PR to original repository.

STEP 11: Add Upstream Repository (To Handle Conflicts)

Command:

git remote add upstream https://github.com/JerryKurata/MachineLearningWithPython.git

Check remotes:

git remote -v

This connects original repository as upstream.

STEP 12: Fetch Upstream Changes

Command:

git fetch upstream

This downloads latest changes from original repository.

STEP 13: Merge Upstream Changes

Command:

git merge upstream/master

If there is conflict, Git will show:

CONFLICT (content): Merge conflict in filename

STEP 14: Resolve Merge Conflict

Open the conflicted file.

Remove conflict markers:

<<<<<<< HEAD

=======

>>>>>>> upstream/master

Keep correct content.

Save the file.

STEP 15: Commit Resolved Conflict

Command:

git add .

git commit -m "Resolved merge conflict"

STEP 16: Push Updated Branch

Command:

git push origin fork-practical

This updates the Pull Request automatically.

Practical 4

1)Open jenkins on terminal -

sudo systemctl start jenkins.service

sudo systemctl status jenkins.service

Browser:

http://localhost:8080

2) login on jenkins

enter username – admin

password – admin123

3) on jenkins

create new item

>> enter item name

>> select freestyle project option

>> ok

job created

4) in job

enter description

>> build steps

>> select execute shell option

code - echo "Welcome"

click on save

>> build now

job created successfully

****for every job save option is must**

5) jenkins to terminal

go to configure

in build step

code : echo "Hello VP "> /tmp/Practical4.txt

>> open terminal

cd /tmp

ls

cat Practical4.txt

>> see output on terminal : Hello VP

6) from terminal to jenkins :

on terminal

code :

sudo bash -c 'echo "Hello Buddy!!" > /var/lib/jenkins/workspace/Practical4/hello.txt'

on jenkins
code : cat hello.txt

>> save

>> build now
see the output for which is created on terminal

Practical 5

Step 1: Create Project Folder

Open terminal and create a folder.

mkdir ai-docker-app

cd ai-docker-app

Step 2: Open Folder in VS Code

Code .

This opens the folder in VS Code.

Your project structure will look like this:

```
ai-docker-app
 ├── app.py
 ├── requirements.txt
 └── Dockerfile
```

Step 3: Create app.py

Create a file app.py and paste this code.

```
from flask import Flask, request, jsonify

app = Flask( name )

@app.route('/')

def home():
```

```
return "AI Model is Running!"
@app.route('/predict', methods=['POST'])
def predict():
data = request.json['number']
result = data * 2
return jsonify({'result': result})
if name == ' main ':
app.run(host='0.0.0.0', port=5000)
```

This is a simple AI API that multiplies a number by 2.

Step 4: Create requirements.txt

Create requirements.txt

```
flask
numpy
```

This installs required Python libraries.

Step 5: Create Dockerfile

Create a file named Dockerfile (no extension).

Paste this:

```
# Use official Python image
FROM python:3.9
# Set working directory
WORKDIR /app
# Copy requirements file
COPY requirements.txt .
# Install dependencies
RUN pip install -r requirements.txt
# Copy project files
COPY . .
```

```
# Expose port
EXPOSE 5000
# Run application
CMD ["python", "app.py"]
```

Step 6: Build Docker Image

Open terminal inside the project folder and run:

```
docker build -t my-ai-app .
```

Output example:

Successfully built XXXXX

Successfully tagged my-ai-app:latest

Step 7: Run Docker Container

Run this command:

```
docker run -p 5000:5000 my-ai-app
```

if not run the use sudo

Output will show Flask server starting.

Step 8: Test Application in Browser

Open browser and go to:

http://localhost:5000

Output:

AI Model is Running!

Step 9: Test Prediction API

Open another terminal and run:

```
curl -X POST http://localhost:5000/predict \
-H "Content-Type: application/json" \
-d "{\"number\": 5}"
```

Output:

{"result":10}

Practical 6

--> Download Docker Compose

sudo curl -L https://github.com/docker/compose/releases/download/v2.27.0/docker-compose-linux-x86_64 \

-o /usr/local/bin/docker-compose

--> Give execute permission

sudo chmod +x /usr/local/bin/docker-compose

--> Verify installation

docker-compose –version

Expected output:

Docker Compose version v2.27.0

--> Run your Hadoop cluster

Now use hyphen version:

docker-compose up -d

(not docker compose)

--> Check containers

docker ps

1. Check Prerequisites

Verify installations:

```
docker --version
docker compose version
python3 --version
```

2.Create Project Directory

```
mkdir ai-docker-app
cd ai-docker-app
```

3. Create Folder Structure

```
mkdir model
mkdir frontend
```

Final structure should look like:

```
ai-docker-app
|
├── docker-compose.yml
|
├── model
|  ├── app.py
|  ├── requirements.txt
| └── Dockerfile
|
└── frontend
 ├── index.html
└── Dockerfile
```

4.Backend (Flask AI Model API)

Go to model folder:

```
cd model
```

Create app.py

```
from flask import Flask
from pymongo import MongoClient
app = Flask(__name__)
client = MongoClient("mongodb://mongodb:27017/")
db = client.testdb
@app.route("/")
def home():
return "AI Model Running"
if __name__ == "__main__":
app.run(host="0.0.0.0", port=5000)
```

Create requirements.txt

```
flask
pymongo
```

Create Dockerfile

```
FROM python:3.9
WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt
COPY app.py .
CMD ["python", "app.py"]
```

5.Frontend Setup

Go back to project folder:

```
cd ..
cd frontend
```

Create index.html

```
<!DOCTYPE html>
<html>
<head>
<title>AI Docker App</title>
</head>
<body>
<h1>AI Docker Application</h1>
<p>Frontend running in Nginx container.</p>
</body>
</html>
```

Create Frontend Dockerfile

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
```

6. Create docker-compose.yml

Go back to root folder:

```
cd ..
```

Create docker-compose.yml

```
version: "3"
services:
mongodb:
image: mongo
container_name: mongo_db
ports:
- "27018:27017"
model:
```

```
build: ./model
ports:
- "5001:5000"
depends_on:
- mongodb

frontend:
build: ./frontend
ports:
- "8081:80"
depends_on:
- model
```

7.Build and Run Containers

Run this command from project root:

sudo docker compose up –build

This will:

• Build model container

• Build frontend container

• Pull MongoDB image

• Start all containers

8.Verify Running Containers

sudo docker ps

Expected containers:

ai-docker-app_model_1

ai-docker-app_frontend_1

mongo_db

9.Test the Application

Test Frontend

Open browser:

http://localhost:8081

Expected output:

AI Docker Application

Frontend running in Nginx container.

Test Backend API

curl http://localhost:5001

Output:

AI Model Running

Test MongoDB

Access container:

sudo docker exec -it mongo_db mongosh

Commands:

show dbs

use testdb

db.users.insertOne({name:"testuser"})

db.users.find()

Exit:

exit

10.View Container Logs

Frontend logs

sudo docker logs ai-docker-app_frontend_1

Model logs

sudo docker logs ai-docker-app_model_1

MongoDB logs

sudo docker logs mongo_db

11.Stop Containers

docker compose down

practical 7

1.Install Docker

sudo apt update

sudo apt install docker.io -y

Start Docker:

sudo systemctl start docker

sudo systemctl enable docker

Add user to Docker group:

sudo usermod -aG docker $USER

newgrp docker

Check installation:

docker --version

docker ps

2.Install kubectl

sudo snap install kubectl --classic

Verify:

kubectl version --client

3.Install Minikube

Download:

curl -LO

https://storage.googleapis.com/minikube/releases/latest/minikube

-linux-amd64

Install:

sudo install minikube-linux-amd64 /usr/local/bin/minikube

Check version:

minikube version

4.Start Kubernetes Cluster

minikube start

Check node:

kubectl get nodes

Expected output:

minikube Ready control-plane

5.Create Project Folder

mkdir ai-kubernetes

cd ai-kubernetes

6.Create AI Application

Create file:

nano app.py

Paste:

```
from flask import Flask, request, jsonify
app = Flask(__name__)
@app.route('/')
def home():

return "AI Model Running Successfully"
```

```
@app.route('/predict', methods=['POST'])
def predict():
data = request.json['number']
result = data * 2
return jsonify({'result': result})
if __name__ == "__main__":
app.run(host="0.0.0.0", port=5000)
```

Save:

CTRL + X

Y

Enter

7.Create Dockerfile

nano Dockerfile

Paste:

```
FROM python:3.10
WORKDIR /app
COPY . .
RUN pip install flask
EXPOSE 5000
CMD ["python","app.py"]
```

8.Build Docker Image

Configure Docker for Minikube:

```
eval $(minikube docker-env)
```

Build image:

```
docker build -t ai-model .
```

Check:

```
docker images
```

9.Create Kubernetes Deployment

nano deployment.yaml

Paste:

```
apiVersion: apps/v1
kind: Deployment
metadata:
name: ai-model-deployment
spec:
replicas: 2
selector:
matchLabels:
app: ai-model
template:
metadata:
labels:
app: ai-model
spec:
containers:
- name: ai-container
image: ai-model
imagePullPolicy: Never
ports:
- containerPort: 5000
```

Apply:

```
kubectl apply -f deployment.yaml
```

Check pods:

```
kubectl get pods
```

10.Create Service

nano service.yaml

Paste:

```
apiVersion: v1
kind: Service
metadata:
name: ai-model-service

spec:
type: NodePort
selector:
app: ai-model
ports:
- port: 5000
targetPort: 5000
nodePort: 30007
```

Apply:

kubectl apply -f service.yaml

Check service:

kubectl get services

11.Access Application

minikube service ai-model-service

Browser will open showing:

AI Model Running Successfully

12.Scale Pods

Increase pods:

kubectl scale deployment ai-model-deployment --replicas=5

Check:

kubectl get pods

You should see 5 pods running.

13.Open Kubernetes Dashboard

minikube dashboard

Practical 8

Install Docker

sudo apt update

sudo apt install docker.io -y

Start Docker

sudo systemctl start docker

sudo systemctl enable docker

Check Docker

docker --version

Install Kubernetes CLI (kubectl)

sudo snap install kubectl --classic

Check installation

kubectl version --client

Install Minikube

curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64

sudo install minikube-linux-amd64 /usr/local/bin/minikube

Check version

minikube version

Start Kubernetes Cluster

minikube start --driver=docker

Verify cluster

kubectl get nodes

Expected output

NAME STATUS ROLES

minikube Ready control-plane

Create Project Folder

mkdir ai-k8s-app

cd ai-k8s-app

Create AI Flask Application

nano app.py

Paste

```
from flask import Flask, request, jsonify
app = Flask(__name__)
@app.route("/")
def home():
return "AI Model Running in Kubernetes"
@app.route("/predict", methods=["POST"])
def predict():
data = request.json["number"]
result = data * 2
return jsonify({"result": result})
if __name__ == "__main__":
app.run(host="0.0.0.0", port=5000)
```

Create Requirements File

nano requirements.txt

Paste

Flask

Create Dockerfile

nano Dockerfile

Paste

FROM python:3.9

WORKDIR /app

COPY . .

RUN pip install flask

EXPOSE 5000

CMD ["python","app.py"]

Build Docker Image

sudo docker build -t ai-model .

Check image

sudo docker images

Load Image into Minikube

minikube image load ai-model

Create Deployment

nano deployment.yaml

Paste

apiVersion: apps/v1

kind: Deployment

```
metadata:
name: ai-app
spec:

replicas: 2
selector:
matchLabels:
app: ai-app
template:
metadata:
labels:
app: ai-app
spec:
containers:
- name: ai-container
image: ai-model
imagePullPolicy: Never
ports:
- containerPort: 5000
resources:
requests:
cpu: "100m"
limits:
cpu: "500m"
```

Deploy

```
kubectl apply -f deployment.yaml
```

Check pods

```
kubectl get pods
```

Create Kubernetes Service

nano service.yaml

Paste

apiVersion: v1

kind: Service

metadata:

name: ai-service

spec:

type: NodePort

selector:

app: ai-app

ports:

- port: 5000

targetPort: 5000

nodePort: 30007

Deploy

kubectl apply -f service.yaml

Check

kubectl get svc

Access Application

minikube service ai-service

Browser opens with:

AI Model Running in Kubernetes

Enable Metrics Server (Required for Autoscaling)

minikube addons enable metrics-server

Verify

kubectl top pods

Example

ai-app-xxxx 30m 100Mi

Create Horizontal Pod Autoscaler

kubectl autoscale deployment ai-app --cpu-percent=50 --min=2 --max=10

Check

kubectl get hpa

Example

NAME CPU% MIN MAX

ai-app 10% 2 10

Generate Load

Run continuously

while true; do curl $(minikube service ai-service --url); done

Observe Auto Scaling

Open another terminal

kubectl get pods -w

Pods increase like

ai-app-1

ai-app-2

ai-app-3

ai-app-4

Monitor Performance

Check CPU

kubectl top pods

Check nodes

kubectl top nodes

Check pod details

kubectl describe pod <pod-name>

Check logs

kubectl logs <pod-name>

Practical 9

1. Create Project Folder

mkdir ai-model cd

ai-model

Create required files:

mkdir model

touch app.py requirements.txt Dockerfile deployment.yaml service.yaml

2. Write Flask Application

Open file:

nano app.py

Paste:

from flask import Flask, request, jsonify import pickle

import numpy as np app =

Flask(name)

model = pickle.load(open('model/model.pkl', 'rb'))

@app.route('/predict', methods=['POST']) def predict():

data = request.json['input']

prediction = model.predict(np.array(data).reshape(1,-1)) return jsonify({'prediction':

prediction.tolist()})

if name == " main ": app.run(host="0.0.0.0", port=5000)

3. Create Requirements File

nano requirements.txt

Paste:

```
flask
numpy
gunicorn
scikit-learn
```

Save and exit.

4. Create AI Model File

Start python:

```
python
```

Run:

```
import pickle
import numpy as np
from sklearn.linear_model import LinearRegression
X = np.array([[1],[2],[3],[4]])
y = np.array([2,4,6,8])
model = LinearRegression()
model.fit(X,y)
pickle.dump(model, open('model/model.pkl','wb'))
exit()
```

5. Test Application Locally

Install packages:

```
pip install -r requirements.txt
```

Run application:

```
python app.py
```

Open new terminal and test:

```
curl -X POST http://localhost:5000/predict \
-H "Content-Type: application/json" \
-d '{"input":[5]}'
```

Expected output:

```
{"prediction":[10]}
```

Stop server: CTRL+C

6. Create Dockerfile

nano Dockerfile

Paste:

```
FROM python:3.12-slim
WORKDIR /app
COPY . /app
RUN pip install --no-cache-dir -r requirements.txt
EXPOSE 5000
CMD ["gunicorn","--bind","0.0.0.0:5000","app:app"]
```

Save.

7. Build Docker Image

```
docker build -t ai-model:latest .
```

Run container:

```
docker run -p 5000:5000 ai-model:latest
```

Test again:

```
curl -X POST http://localhost:5000/predict \
-H "Content-Type: application/json" \
-d '{"input":[5]}'
```

Stop container: CTRL+C

8. Push Image to Docker Hub

Login:

```
docker login
```

Tag image:

```
docker tag ai-model:latest tanujalokhande10/ai-model:latest
```

Push image:

```
docker push tanujalokhande10/ai-model:latest
```

9. Create Kubernetes Deployment

```
nano deployment.yaml
```

Paste:

```
apiVersion: apps/v1
kind: Deployment
metadata:
name: ai-model-deployment
spec:
replicas: 2
selector:
matchLabels:
app: ai-model
template:
metadata:
labels:
app: ai-model
spec:
containers:
- name: ai-model
image: tanujalokhande10/ai-model:latest
```

ports:

- containerPort: 5000

Save.

10. Create Kubernetes Service

nano service.yaml

Paste:

apiVersion: v1

kind: Service

metadata:

name: ai-model-service

spec:

type: NodePort

selector:

app: ai-model

ports:

- protocol: TCP

port: 5000

targetPort: 5000

nodePort: 30007

Save.

11. Deploy to Kubernetes

kubectl apply -f deployment.yaml

kubectl apply -f service.yaml

12. Verify Deployment

Check pods:

kubectl get pods

Expected:

ai-model-deployment-xxxxx 1/1 Running

ai-model-deployment-yyyyy 1/1 Running

Check service:

kubectl get svc

Example output:

ai-model-service NodePort 5000:30007

13. Get Node IP

kubectl get nodes -o wide

Example:

INTERNAL-IP

192.168.49.2

14. Test API from Kubernetes

```
curl -X POST http://192.168.49.2:30007/predict \
-H "Content-Type: application/json" \
-d '{"input":[5]}'
```

Expected response:

{"prediction":[10]}

Final Working Architecture

Flask AI App

↓

Docker Container

↓

Docker Hub

↓

Kubernetes Deployment

↓

NodePort Service

↓

AI Model API Accessible

Important commands teachers usually check

kubectl get pods

kubectl get svc

kubectl get nodes -o wide